FORMOSA CRYPTO

Completing the Chain:

Verified Implementations of Hash-Based Signatures and Their Security

Manuel Barbosa François Dupressoir **Rui Fernandes**
Andreas Hülsing Matthias Meijers Pierre-Yves Strub

GitHub Repository

Paper

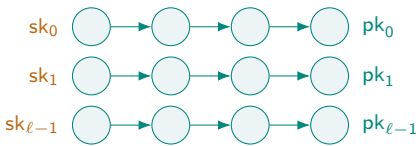
CHES 2026

XMSS: Overview

- XMSS [BDH11] is a stateful hash-based signature scheme.
- Security relies on the security of the underlying hash function.
- XMSS combines a one-time signature scheme (WOTS^+) with a Merkle tree.
  - The root of the Merkle tree is the public key of the XMSS instance.
  - Each leaf corresponds to a (compressed) WOTS^+ public key.
  - A Merkle tree of height h has 2^h leaves, and thus allows signing 2^h messages.

WOTS$^+$: Key Generation

1. Generate ℓ secret n-byte values $\mathsf{sk}_0, \ldots, \mathsf{sk}_{\ell-1}$ (pseudorandomly)
2. Each value starts a hash chain: apply the chaning function $w-1$ steps to obtain
3. The chaining function is defined as $c_k^i(x, \mathbf{r}) = f_k(c_k^{i-1}(x, \mathbf{r}) \oplus r_i)$, with $c_k^0(x, \mathbf{r}) = x$, where $\mathbf{r}$ are randomizers and k is keying parameter for the hash function.



Example: $w = 4$

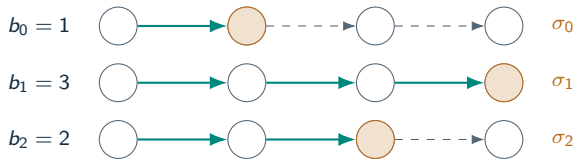
Secret key: $\mathsf{SK} = (\mathsf{sk}_0, \ldots, \mathsf{sk}_{\ell-1})$

Public key: $\mathsf{PK} = (\mathsf{pk}_0, \ldots, \mathsf{pk}_{\ell-1})$

n: element size in bytes; w: Winternitz parameter; ℓ: number of message and checksum digits.

WOTS^+: Signature Generation & Verification

1. Encode the message as base-w digits $m_0, \ldots, m_{\ell_1 - 1}$
2. Append the base-w digits of the checksum $C = \sum_{j=0}^{\ell_1 - 1}(w - 1 - m_j)$, giving ℓ digits $b_0, \ldots, b_{\ell - 1} \in \{0, \ldots, w - 1\}$.
3. The signature is $(\sigma_0, \ldots, \sigma_{\ell - 1})$, where $\sigma_j = c_k^{b_j}(\mathsf{sk}_j, \mathbf{r})$.
4. To verify a signature, it suffices to confirm that completing the chains permits reconstructing the WOTS^+ public key.



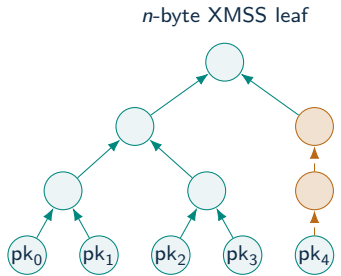
Example: $w = 4$; gold nodes form the signature.

Each WOTS^+ keypair signs at most one message.

L-Tree: Compressing a WOTS$^+$ Public Key

Input: ℓ public-key elements, each n bytes.

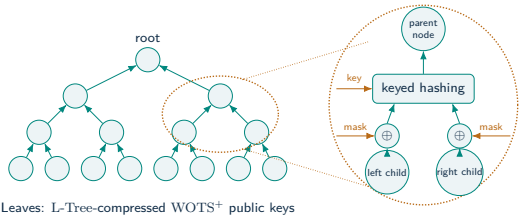
1. Hash adjacent pairs with randomized hashing (hash keys and masks are derived from `pub_seed` and the L-Tree address).
2. If a level has an odd number of nodes, carry the last node to the next level unchanged.
3. Repeat until a single n-byte node remains.



Example: $\ell = 5$
Gold dashed arrows: carry unchanged

XMSS: **Key Generation**

1. Sample n-byte seeds `sk_seed`, `sk_prf`, and `pub_seed`; initialize the signing index $i = 0$.
2. Derive WOTS^+ secret keys via a PRF keyed by `sk_seed`; compute and compress (via $\mathrm{L\text{-}Tree}$) the 2^h public keys.
3. Build the XMSS tree by hashing pairs of sibling nodes together to obtain the root. A PRF keyed by `pub_seed` derives hash keys and bitmasks using the node address.



Secret key: $\mathrm{SK} = (i, \mathtt{sk_seed}, \mathtt{sk_prf}, \mathrm{PK})$ **Public key:** $\mathrm{PK} = (\mathrm{root}, \mathtt{pub_seed})$

`sk_prf`: PRF key for message-hashing randomizers during signing.

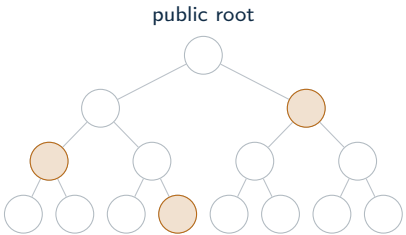
XMSS: **Signature Generation**

1. Set the index i to the current secret key index: (indicates the one-time WOTS^+ key used).
2. Update the index in the secret key to prevent key reuse.
3. Derive a pseudorandom value R using a PRF keyed by `sk_prf`, then compute the message digest using R.
4. Sign the digest with the selected WOTS^+ secret key.
5. Compute the authentication path for leaf i: it authenticates the corresponding WOTS^+ public key in the XMSS tree.
6. The signature is $\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathrm{auth})$

XMSS: **Signature Verification**

Inputs: message M, PK $= (\text{root}, \text{SEED})$, $\sigma = (i, R, \sigma_{\text{WOTS}^+}, \text{auth}_i)$, with $0 \leq i < 2^h$.

1. Reconstruct the message digest using R.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover the WOTS^+ public key.
3. L-Tree compression of the WOTS^+ public key gives us the i-th leaf of the XMSS tree.
4. Use the authentication path to compute the root of the XMSS tree.
5. Accept if the reconstructed root equals root in the public key.



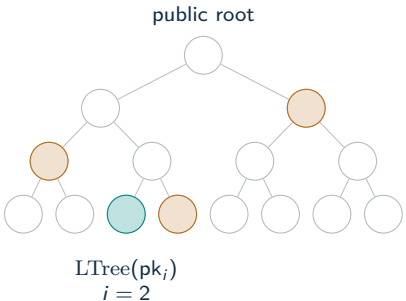
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS^+}}, \mathsf{auth}_i)$, with $0 \leq i < 2^h$.

1. Reconstruct the message digest using R.
2. Complete the $\mathrm{WOTS^+}$ chains from $\sigma_{\mathrm{WOTS^+}}$ to recover the $\mathrm{WOTS^+}$ public key.
3. L-Tree compression of the $\mathrm{WOTS^+}$ public key gives us the i-th leaf of the XMSS tree.
4. Use the authentication path to compute the root of the XMSS tree.
5. Accept if the reconstructed root equals root in the public key.



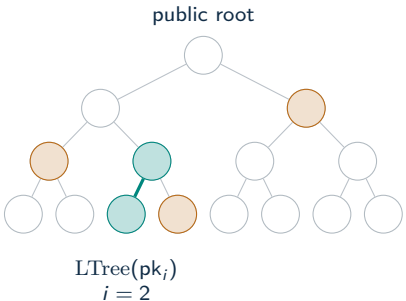
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS^+}}, \mathsf{auth}_i)$, with $0 \leq i < 2^h$.

1. Reconstruct the message digest using R.
2. Complete the $\mathrm{WOTS^+}$ chains from $\sigma_{\mathrm{WOTS^+}}$ to recover the $\mathrm{WOTS^+}$ public key.
3. L-Tree compression of the $\mathrm{WOTS^+}$ public key gives us the i-th leaf of the XMSS tree.
4. Use the authentication path to compute the root of the XMSS tree.
5. Accept if the reconstructed root equals root in the public key.



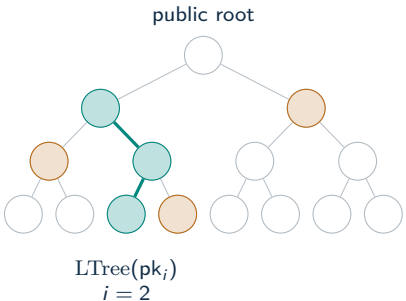
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS^+}}, \mathsf{auth}_i)$, with $0 \leq i < 2^h$.

1. Reconstruct the message digest using R.
2. Complete the $\mathrm{WOTS^+}$ chains from $\sigma_{\mathrm{WOTS^+}}$ to recover the $\mathrm{WOTS^+}$ public key.
3. L-Tree compression of the $\mathrm{WOTS^+}$ public key gives us the i-th leaf of the XMSS tree.
4. Use the authentication path to compute the root of the XMSS tree.
5. Accept if the reconstructed root equals root in the public key.



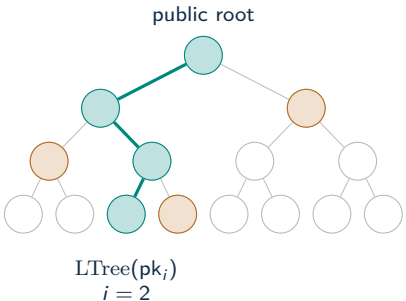
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathsf{auth}_i)$, with $0 \leq i < 2^h$.

1. Reconstruct the message digest using R.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover the WOTS^+ public key.
3. L-Tree compression of the WOTS^+ public key gives us the i-th leaf of the XMSS tree.
4. Use the authentication path to compute the root of the XMSS tree.
5. Accept if the reconstructed root equals root in the public key.



Gold: supplied authentication siblings.

Teal: reconstructed path.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- $\mathrm{XMSS}^{\mathrm{MT}}$ replaces the single XMSS Merkle tree with a hypertree of d layers; each tree has height h/d.
- **Key generation:** compute the root of the top tree in the hypertree to obtain the public root.
- **Signature generation:** include one WOTS^{+} signature and its authentication path for each of the d layers. At each upper layer $j = 1, \ldots, d-1$, sign the root of the tree below; skip the message-digest computation.
- Allows more signatures while keeping running times manageable, at the expense of increased signature size.

Jasmin [ABB+17, ABB+20]

- High-level assembly language designed for cryptography implementations
  - High-level zero-cost abstractions: variables, arrays
  - Control over low-level details (e.g. instruction selection, loop unrolling, and whether variables are stored on the stack or in registers)
- Applicability to the development of high-speed, high-assurance cryptographic primitives has been demonstrated through its application to SHA-3 [ABB+19] and ML-KEM [AOB+24, ABB+23].
- Suported architectures: x86_64, ARMv7M, ARMv8-A, RISC-V

Jasmin [ABB+17, ABB+20]

- Stack zeroization [OBG+24]
- (Static) Safety checker
- Compiler correctness proved in Rocq
- Extraction to EasyCrypt
- Speculative constant-time:
  - Spectre v1 [SBG+23] and other spectre variants [ABC+25]
  - Compilation preserves SCT [ABB+25].

Compiler Correctness & Constant-Time Preservation

Compilation preserves the semantics of *safe* source programs and constant-time behavior: functional correctness and constant-time guarantees established at source level carry to the assembly level.

EasyCrypt

- Interactive proof assistant used for cryptographic security proofs and reasoning about program correctness.
- Supports several logics:
  - **Ambient Logic (AL):** structures proofs and combines results from the other logics.
  - **Hoare Logic (HL):** proves properties of a single program using preconditions and postconditions.
  - **Probabilistic Hoare Logic (pHL):** proves bounds on the probability of a program satisfying a postcondition.
  - **Probabilistic Relational Hoare Logic (pRHL):** proves equivalences between two probabilistic programs.

Verification Workflow

EasyCrypt

Jasmin

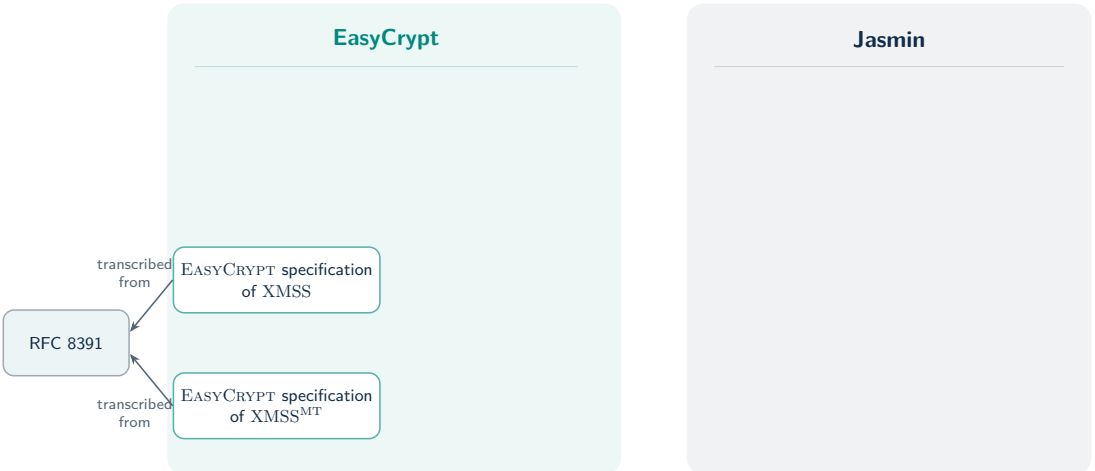
Verification Workflow

RFC 8391

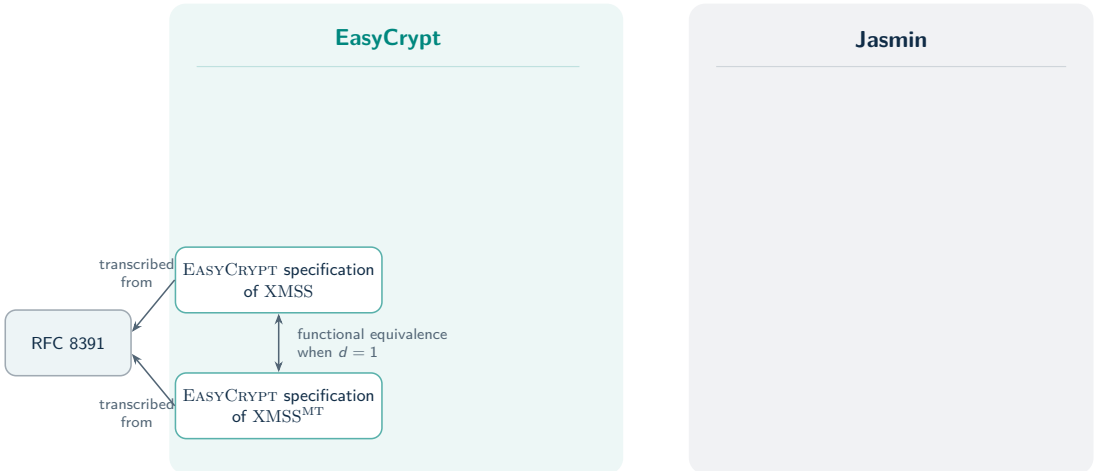
EasyCrypt

Jasmin

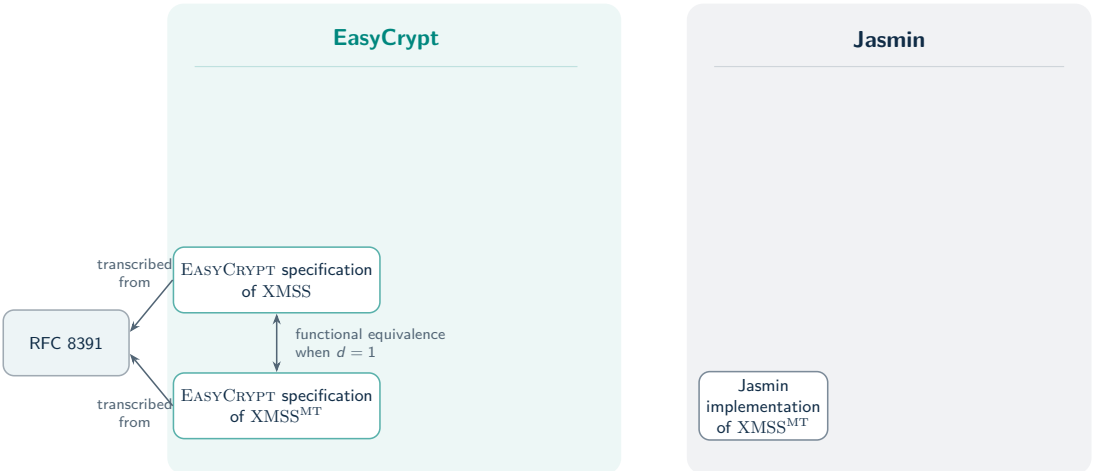
Verification Workflow



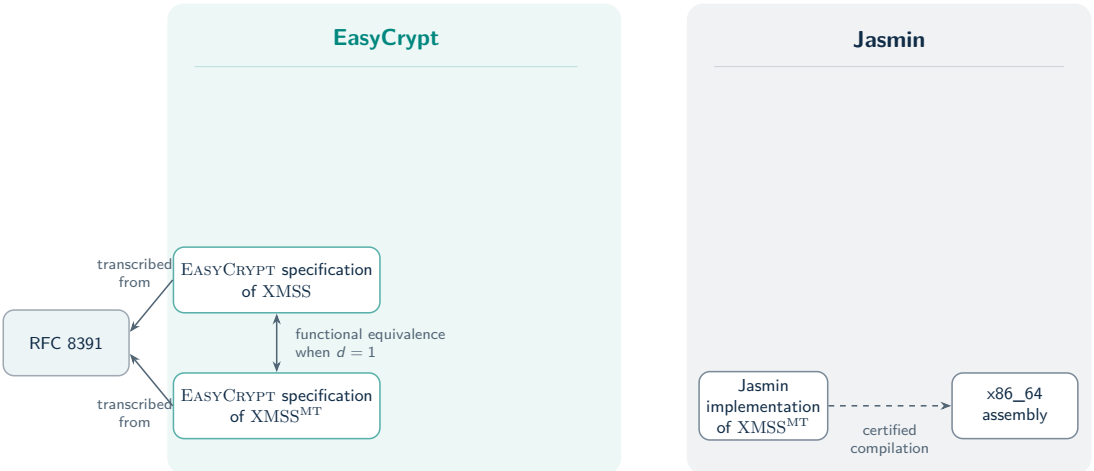
Verification Workflow



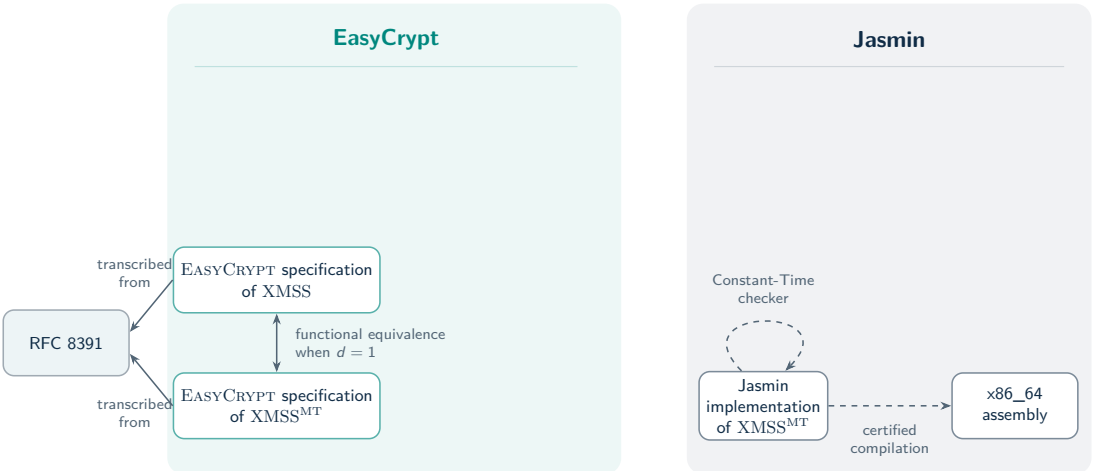
Verification Workflow



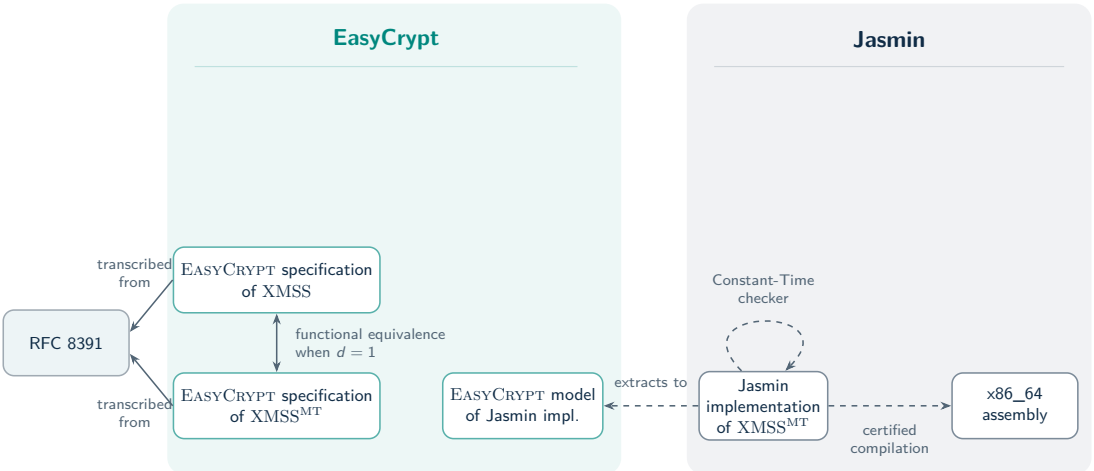
Verification Workflow



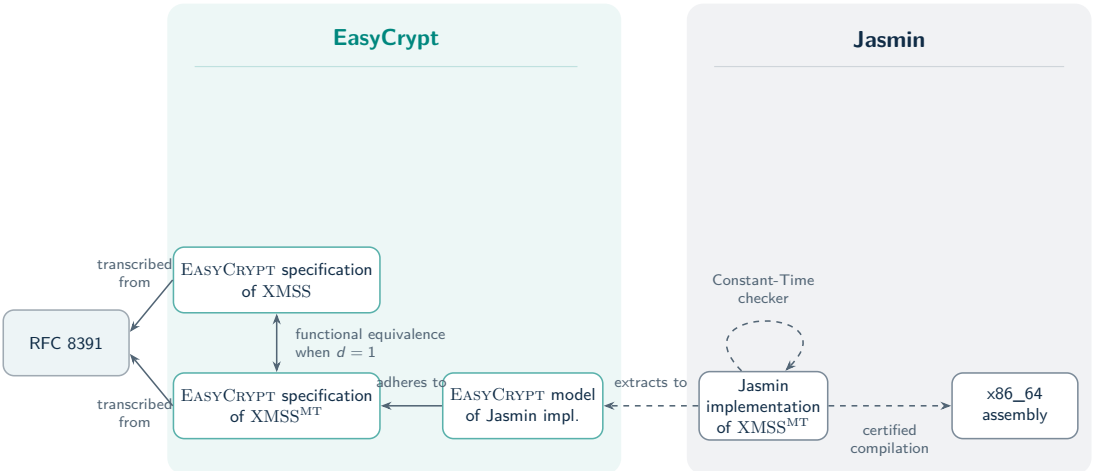
Verification Workflow



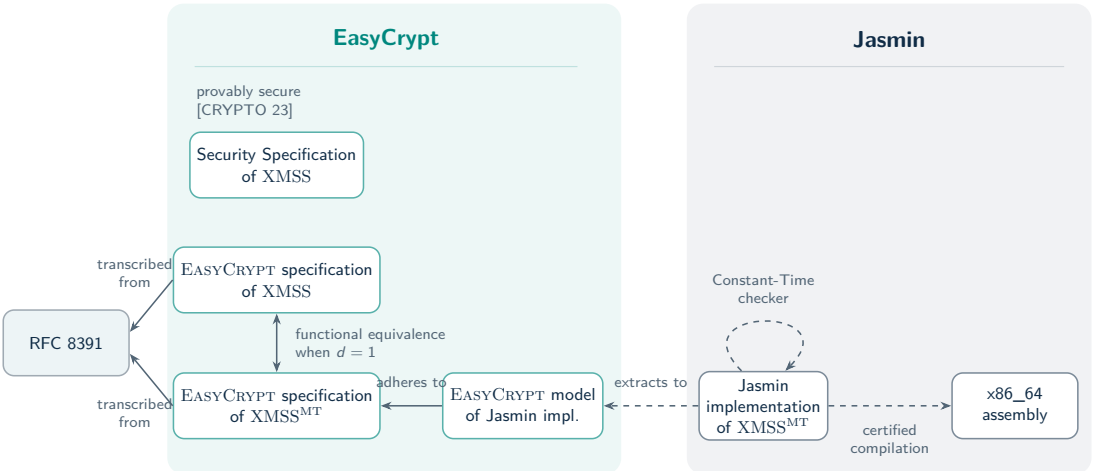
Verification Workflow



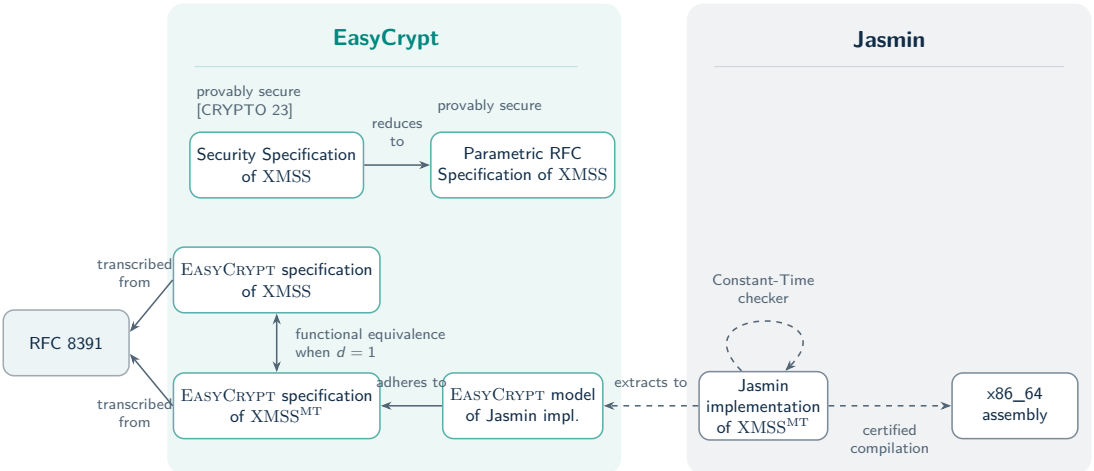
Verification Workflow



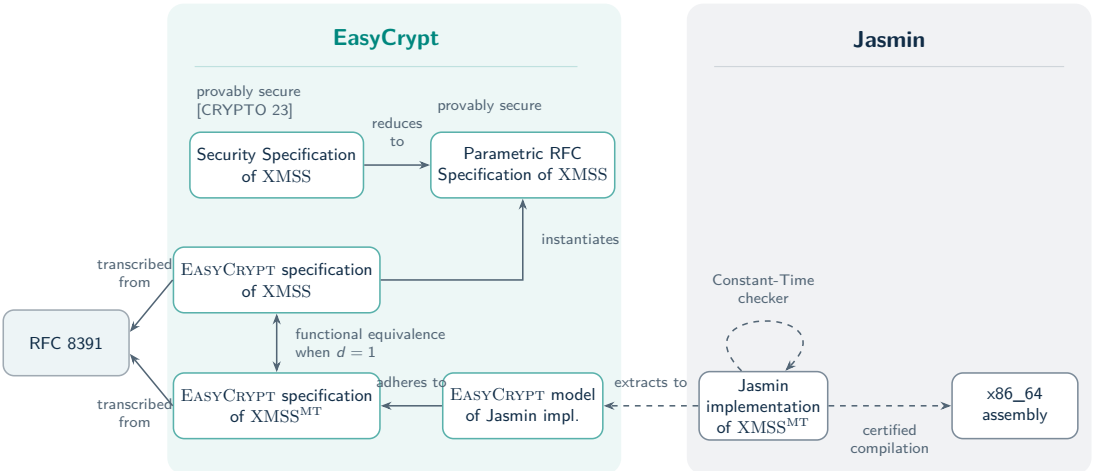
Verification Workflow



Verification Workflow



Verification Workflow



Trusted Computing Base

- Correctness of the SHA-256 implementation is outside the scope of the proof

```
op Hash : W8.t list -> nbytes. (* Abstract operator *)

axiom hash_128 (x : W8.t Array128.t):
  phoare[
    M(Syscall).hash_plen_3n____sha256:
    arg.`2 = x
    ==>
    to_list res = NBytes.val (Hash (to_list x))
  ] = 1%r.
```

Figure: EasyCrypt axiom stating the correctness of SHA-256.

Trusted Computing Base

- Jasmin compiler: unverified OCaml code (parsing and typing)
- Jasmin extraction to EasyCrypt
- EasyCrypt codebase
- SMT Solvers (Z3, CVC4 and Alt-Ergo)

Implementation: Overview

- $\mathrm{XMSS}^{\mathrm{MT}}$ implementation written in Jasmin targeting x86_64, based on the C reference implementation[1], prioritizing readability and closeness to RFC 8391.
- **SHA-256 from libjade**[2] (the C reference uses OpenSSL).
- **Fixed-size data:** stack arrays with sizes known at compile time; a separate hash function for each input length.
- **Variable-size data:** raw pointers with explicit loads and stores, e.g. for the message to be signed.
- **Arrays:** array arguments must be disjoint.

[1] https://github.com/XMSS/xmss-reference
[2] https://github.com/formosa-crypto/libjade

Implementation: Authentication Paths

- **TreeHash algorithm:** Used to compute the root of a subtree: keep intermediate nodes on a stack; merge the top two nodes whenever they have the same height.
- **Jasmin impl. follows RFC 8391:** compute each node of the authentication-path with a separate TreeHash call on the corresponding subtree.
- **C reference:** compute the root and authentication path together in a single TreeHash traversal.
- **Tradeoff**: Closer correspondence to the RFC simplifies the correctness proof, but increases execution time.

Implementation: Constant-Time and Zeroization

- **Constant-time validated with Jasmin's CT checker.** Annotate public values loaded from memory (e.g. message length): memory loads are secret by default, and need to be explicitly marked as public.
- **Stack zeroization on return from exported functions:** compiler-generated code clears the stack, ensuring that no secret data remains on the stack after function return.
  - Loop zeroization adds less than 0.4% overhead for all three operations. Unrolling adds about 30 KB of code and is slightly slower in these benchmarks.

Implementation: Performance

	C Ref	Jasmin (abs, %)	loop zeroization (abs, %)	unrolled zeroization (abs, %)
Keygen	10092.74	8263.54 (-18.12%)	8293.61 (+0.36%)	8312.35 (+0.59%)
Sign	20251.01	24775.17 (+22.34%)	24872.05 (+0.39%)	24927.53 (+0.61%)
Verify	10.35	8.47 (-18.14%)	8.48 (+0.13%)	8.95 (+5.69%)

Table: Performance Comparison between C and Jasmin implementations in 10^6 clock cycles (column #2). Zeroization overhead w.r.t. Jasmin baseline (columns #3 and #4).

- **Keygen and verify:** about 18% fewer cycles, driven by the faster SHA-256 implementation.
- **Sign:** about 22% more cycles, due to separate TreeHash calls for authentication-path nodes.

Functional Correctness Proof

- **Goal**: Prove that the Jasmin implementation of $\mathrm{XMSS}^{\mathrm{MT}}$ adheres to the RFC-like EasyCrypt specification.
- Extract an EasyCrypt representation of the Jasmin program using the Jasmin compiler. Program equivalence is then expressed via **pRHL** statements.
- We assume the functional correctness of the underlying SHA-256 implementation.

Functional Correctness: Relating Representations

- **Specification:** records for keys and signatures.
- **Implementation:** byte arrays and raw pointers.
- Encoding and decoding operators connect these representations

```
lemma xmss_kg_correct :
    n = XMSS_N /\ (* remaining parameters ommited *) =>
    equiv [
      M(Syscall).__xmssmt_core_keypair ~ XMSS_MT_PRF.kg :
      true
      ==>
      res{1}.`1 = DecodePkNoOID res{2}.`2 /\
      res{1}.`2 = DecodeSkNoOID res{2}.`1
    ].
```

Figure: EasyCrypt lemma stating the correctness of the key generation algorithm.

Functional Correctness Proof: Reasoning About Memory

- Raw pointers require explicit reasoning about address arithmetic and the contents of memory (e.g. pointers don't overlap and do not overflow the 64-bit address space).

```
pred valid_ptr_i (p : W64.t) (o : int) =
  0 <= o =>
    0 <= to_uint p /\ to_uint (p) + o < W64.modulus.

pred disjoint_ptr (p1 l1 p2 l2 : int) =
  forall (k1 : int), 0 <= k1 < l1 =>
    forall (k2 : int), 0 <= k2 < l2 =>
      p1 + k1 <> p2 + k2.
```

Figure: Pointer predicates in EasyCrypt.

TreeHash

- Proving functional correctness of TreeHash was the most demanding part of the proof
- Computes the root of a subtree from its leaves:
  - **Security Specification**: Expressed as a recursive function over an inductive data type for trees.
  - **RFC Specification**: Iterative function that makes use of a stack for storing intermediate nodes.
- Rather than proving that implementation and specification are accessing the stack within bounds, we prove that they perform identical accesses (which could be within bounds or out of bounds), and prove that they produce the same result.

QUESTIONS?

References I

[ABB+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub.

Jasmin: High-assurance and high-speed cryptography.

In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1807–1823. ACM Press, October / November 2017.

[ABB+19] José Bacelar Almeida, Cecile Baritel-Ruet, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Alley Stoughton, and Pierre-Yves Strub.

Machine-checked proofs for cryptographic standards: Indifferentiability of Sponge and secure high-assurance implementations of SHA-3.

In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1607–1622. ACM Press, November 2019.

References II

[ABB+20] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub.

The last mile: High-assurance and high-speed cryptographic implementations.

In *2020 IEEE Symposium on Security and Privacy*, pages 965–982. IEEE Computer Society Press, May 2020.

[ABB+23] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, Antoine Séré, and Pierre-Yves Strub.

Formally verifying Kyber episode IV: Implementation correctness.

IACR TCHES, 2023(3):164–193, 2023.

[ABB+25] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Grégoire, and Vincent Laporte.

Preservation of speculative constant-time by compilation.

Proc. ACM Program. Lang., 9(POPL):1293–1325, 2025.

References III

[ABC+25] Santiago Arranz-Olmos, Gilles Barthe, Chitchanok Chuengsatiansup, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Peter Schwabe, Yuval Yarom, and Zhiyuan Zhang.

Protecting cryptographic code against spectre-rsb: (and, in fact, all known spectre variants).

In Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 933–948. ACM, 2025.

[AOB+24] José Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub.

Formally verifying Kyber - episode V: Machine-checked IND-CCA security and correctness of ML-KEM in EasyCrypt.

In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 384–421. Springer, Cham, August 2024.

References IV

[BDH11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing.
XMSS - A practical forward secure signature scheme based on minimal security assumptions.
In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings*, volume 7071 of *Lecture Notes in Computer Science*, pages 117–129. Springer, 2011.

[OBG+24] Santiago Arranz Olmos, Gilles Barthe, Ruben Gonzalez, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, and Peter Schwabe.
High-assurance zeroization.
IACR TCHES, 2024(1):375–397, 2024.

[SBG+23] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean.
Typing high-speed cryptography against spectre v1.
In *2023 IEEE Symposium on Security and Privacy*, pages 1094–1111. IEEE Computer Society Press, May 2023.